第2章练习题：习题集2.8

2.10　2.21　2.22　2.34　2.35

第3章作业：严蔚敏教材配套习题集

3. 5　　3. 6　　3. 10　　3. 21　　3. 29　　3. 32

数据结构课程的内容

- 数据结构
  - 逻辑结构
    - 线性结构（线性表、栈 、队、串、数组）
    - 非线性结构
      - 树结构
      - 图结构
  - 物理（存储）结构
    - 顺序结构
    - 链式结构
    - 索引结构
    - 散列结构
  - 数据运算
    - 插入运算
    - 删除运算
    - 修改运算
    - 查找运算
    - 排序运算

第三章 栈和队列

3.1 栈（Stack）

1. 定义
2. 逻辑结构
3. 存储结构
4. 运算规则
5. 实现方式

3.2 队列（Queue）

1. 定义
2. 逻辑结构
3. 存储结构
4. 运算规则
5. 实现方式

3.1 栈

即栈顶

1. 定义　限定只能在表的一端进行插入和删除运算的线性表。

2. 逻辑结构　与线性表相同，仍为一对一(**1:1**)关系。

3. 存储结构　用顺序栈或链栈存储均可，但以顺序栈更常见

4. 运算规则　只能在栈顶运算，且访问结点时依照**后进先出（LIFO）或先进后出（FILO）的原则。**

5. 实现方式　关键是编写入栈和出栈函数，具体实现依顺序栈或链栈的存储结构有别而不同。

基本操作有：**建栈、判断栈满或栈空、入栈、出栈、读栈顶元素值，等等。**

栈 是仅在**表尾**进行插入、删除操作的线性表。

表尾(即 a_n 端)称为**栈顶** /top ； 表头(即 a_1 端)称为**栈底**/base

例如： 栈 S= (a_1 , a_2 , a_3 , ……,a_{n-1} , a_n)

a_1称为栈底元素

a_n称为栈顶元素

强调：**插入和删除都只能在表的一端（栈顶）进行！**

插入元素到栈顶的操作，称为**入栈**。

从栈顶删除最后一个元素的操作，称为**出栈**。

想一想：要从栈中取出a_1，应当如何操作?

例**1**（严题集3.1）一个栈的输入序列为**1,2,3**，若在入栈的过程中允许出栈，则可能得到的出栈序列是什么？

答：可以通过穷举所有可能性来求解：

① 1入1出， 2入2出，3入3出， 即123；

② 1入1出， 2、3入，3、2出， 即132；

③ 1、2入，2出， 3入3出， 即231；

④ 1、2入，2、1出，3入3出， 即213；

⑤ 1、2、3入，3、2、1出， 即321；

合计有5种可能性。

例2：一个栈的输入序列是12345，若在入栈的过程中允许出栈，则栈的输出序列43512可能实现吗？12345的输出呢？

答：

43512不可能实现，主要是其中的12顺序不能实现；

12345的输出可以实现，每压入一数便立即弹出即可。

计算机系2001年考研题

例3：

设依次进入一个栈的元素序列为c，a，b，d，
则可得到出栈的元素序列是：

A）a，b，c，d　　B）c，d，a，b
C）b，c，d，a　　D）a，c，d，b

答：A）、D）可以，B）、C）不行。

讨论：有无通用的判别原则？

有！若输入序列是 …,P_j…P_k…P_i …(P_j<P_k<P_i)，
一定不存在这样的输出序列 …,P_i…P_j…P_k …

即对于输入序列1，2，3，不存在输出序列3，1，2

Q1：堆栈是什么？它与一般线性表有什么不同？

堆栈是一种特殊的线性表，它只能在表的一端（即栈顶）进行插入和删除运算。

与一般线性表的区别：仅在于运算规则不同。

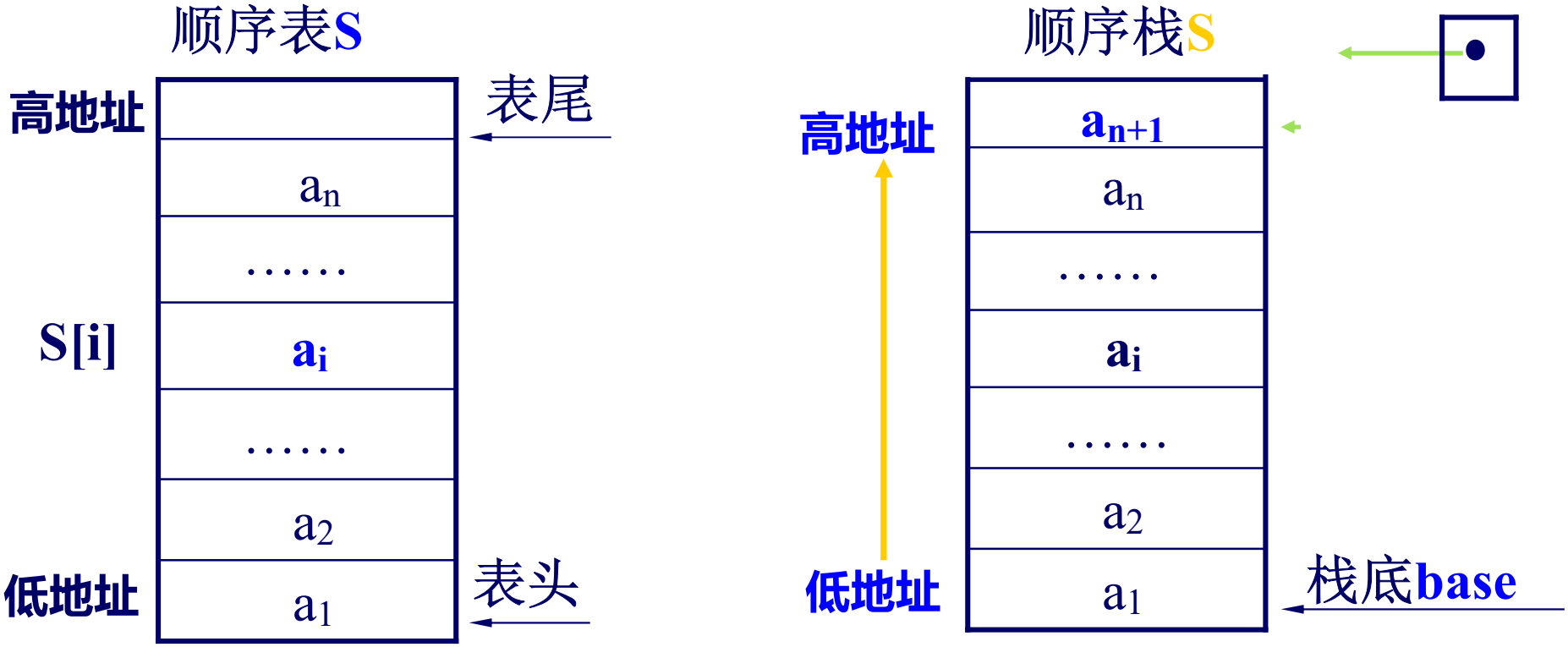
一般线性表	堆栈
逻辑结构：1:1	逻辑结构：1:1
存储结构：顺序表、链表	存储结构：顺序栈、链栈
运算规则：随机存取	运算规则：后进先出(LIFO)

“进” = 插入=压入=PUSH (a_{n+1})

“出” = 删除=弹出 =POP(a_n)

Q2：顺序表和顺序栈的操作有何区别？

以线性表 $S=(a_1, a_2, \ldots, a_{n-1}, a_n)$为例



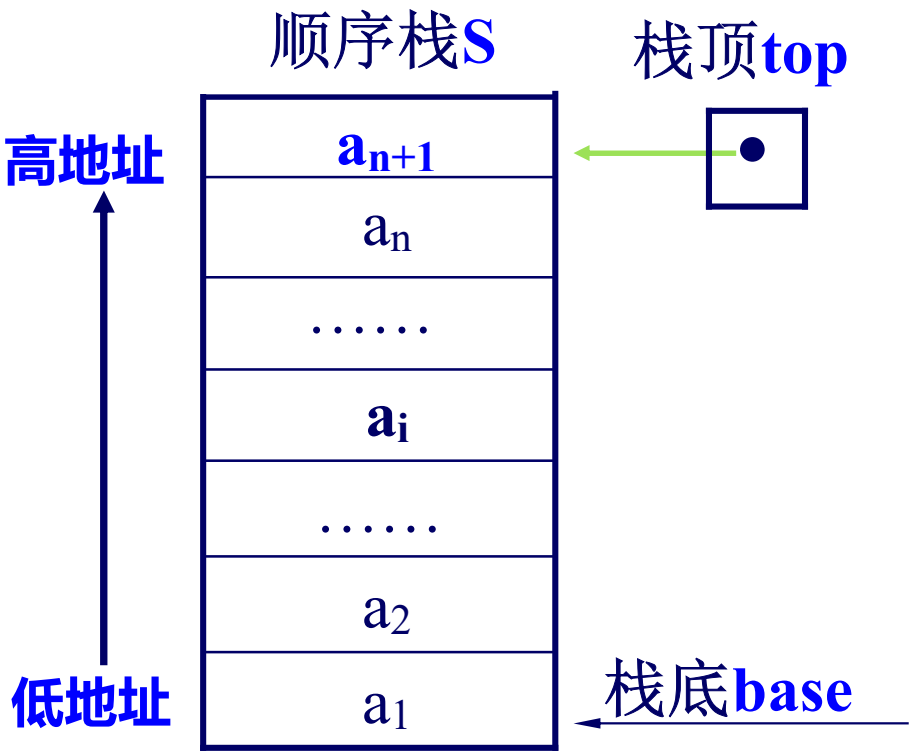
写入：S[i]= a_i

读出： e= S[i]

压入(PUSH)：S[top++]=a_{n+1}

弹出(POP)： e=S[--top]

前提：一定要预设栈顶指针top



栈不存在的条件： base=NULL;

栈为空 的条件 ： base=top;

栈满的条件 ： top-base=stacksize;

Q3：什么叫“向上生成”的栈？ “向下生成”又是何意？

若入栈动作使地址向高端增长，称为“向上生成”的栈；

若入栈动作使地址向低端增长，称为“向下生成”的栈；

对于向上生成的堆栈：

入栈口诀：堆栈指针**top**“先压后加”： S[top++]=a_{n+1}

出栈口诀：堆栈指针**top**“先减后弹”： e=S[--top]

Q4：为什么要设计堆栈？它有什么独特用途？

1. 调用函数或子程序非它莫属；
2. 递归运算的有力工具；
3. 用于保护现场和恢复现场；
4. 简化了程序设计的问题。

下面再用3个例子来帮助理解堆栈和递归：

什么叫递归？

一个直接或间接调用自身的函数被称为是递归（recursion）的。

例4(第1章自测卷)： 写出下面C程序段的执行结果。

```
main( )
{ int n;
long y;
n=5;
y=fact(n);
printf("%d,%ld\n",n,y); }
```

```
long int fact(n)
int n;
{  long f;
if(n>1)f=n*fact(n-1);
  else f=1;
return(f); }
```

运行结果为： 5，120

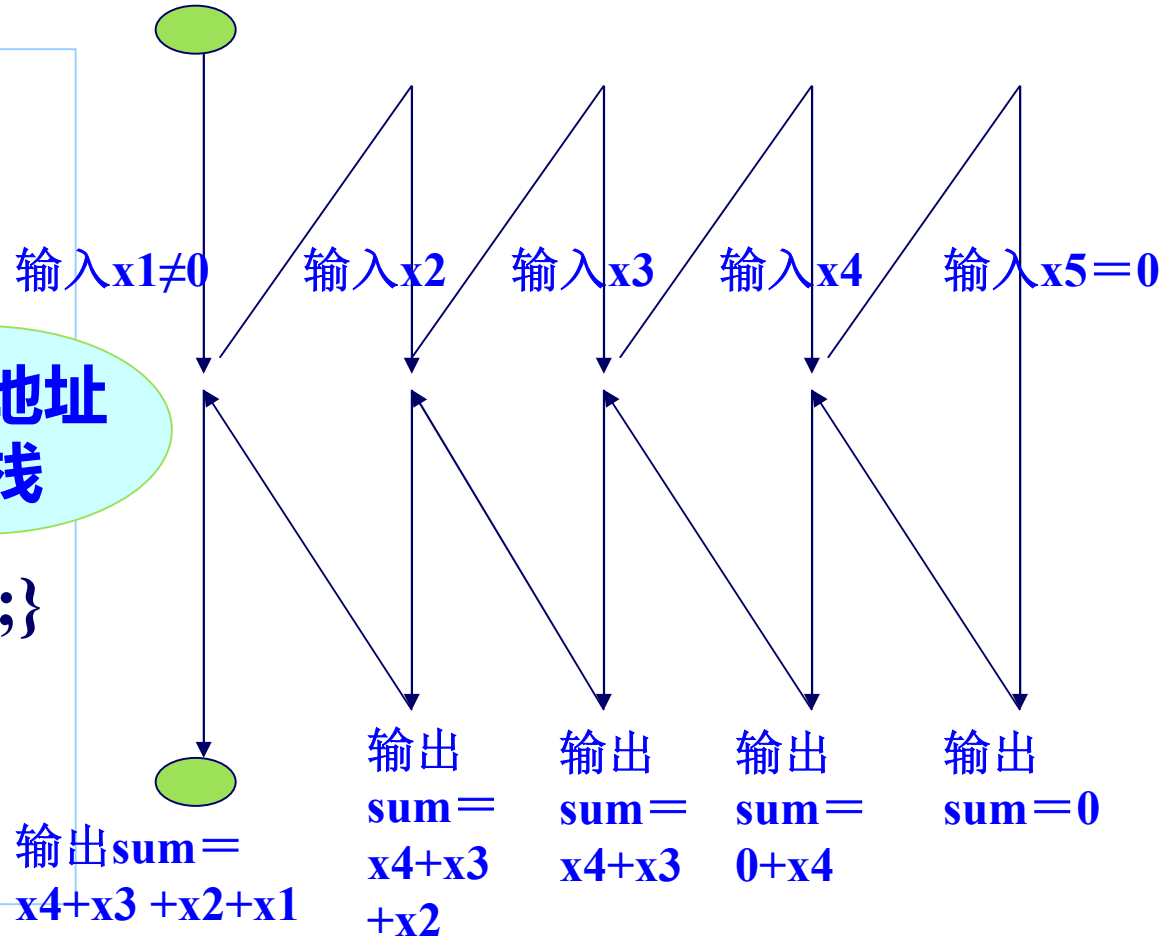
用递归方式计算阶乘N!

例5（严题集3.10③）阅读下列递归过程，说明其功能。

```
void test(int &sum){
int x;
scanf(x);
if(x==0)sum=0;
else{test(sum);sum+=x;}
printf(sum);
}
```

断点地址入栈

注意：最先输入的数据 x_1 最后才被累加



程序功能：对键盘输入数据求和，直到输入0结束

编制递归算法要注意些什么？

- 递归进行是有条件的。一般常把判断语句加在递归语句以前。
- 递归的最底层应该有返回值，以供上层递归的调用。否则会死循环。
- 参量的初始化应该在递归以前。
- 递归调用需要利用堆栈。

每次调用要把本次调用的参数和局部变量保存在栈顶。每次从下一层调用返回到上一层调用时，从栈顶恢复本层调用的参数和局部变量的值。

例6： 设正整数a的前驱为**PRIOR(a)**，后继为**NEXT(a)**，用递归算法计算a+b。

清华考研题

解：首先要对加法算法进行恰当的描述。肯定不能用简单的加法语句，要考虑如何用给出的两个特定函数来实现a+b?

思路：若用数轴来描述a+b，是两条线段之和，也可以看作是a不断往“0”移动，b不断往相反方向移动的过程。

当a移到0时，b指向的位置即为a+b。

(a可视为一减计数器，前移过程中不断递减，而b的后移则是不断加1的过程)

平常我们可以用循环来实现，但根据本题题意不能用循环，而要用递归。

设计要点有二：①递归算法的形式化描述；②不能无限制递归，一定要有终止条件。

```
int  addint (int a, int b)
{  if(a>b)a←→b ;
   return ( add(a, b) )
}
```

上层再加一个函数！

```
int add( int a , int b );
if a= =0 return (b);
else return( add(prior(a), next(b)) );
```

~~if (a>b) a←→b~~

讨论算法复杂度：若总以a为基准，那么当a很大而b很小时则速度会慢很多，应当用较小的数作为a（减1计数器）才合理。

应当增加语句，判断a>b？ **但加在递归结构体内好不好？**

有冗余！在递归开始之前判断一次足矣，速度会快很多。

动态数组

顺序栈的存储表示（教材P46)：

```
#define    STACK-INIT-SIZE    100  //存储空间初始分配量
#define    STACKINCREMENT    10   //存储空间分配增量
typedef    struct {
           SElemType    *base;    //栈的基址即栈底指针
           SElemType    *top;     //栈顶指针
           int          stacksize; //当前分配的空间
          }SqStack;
```

顺序栈和链栈的基本操作 (本节重点)

栈的抽象数据类型定义：

(教材P41-42)

ADT Stack{

数据对象：D=……

数据关系：R=……

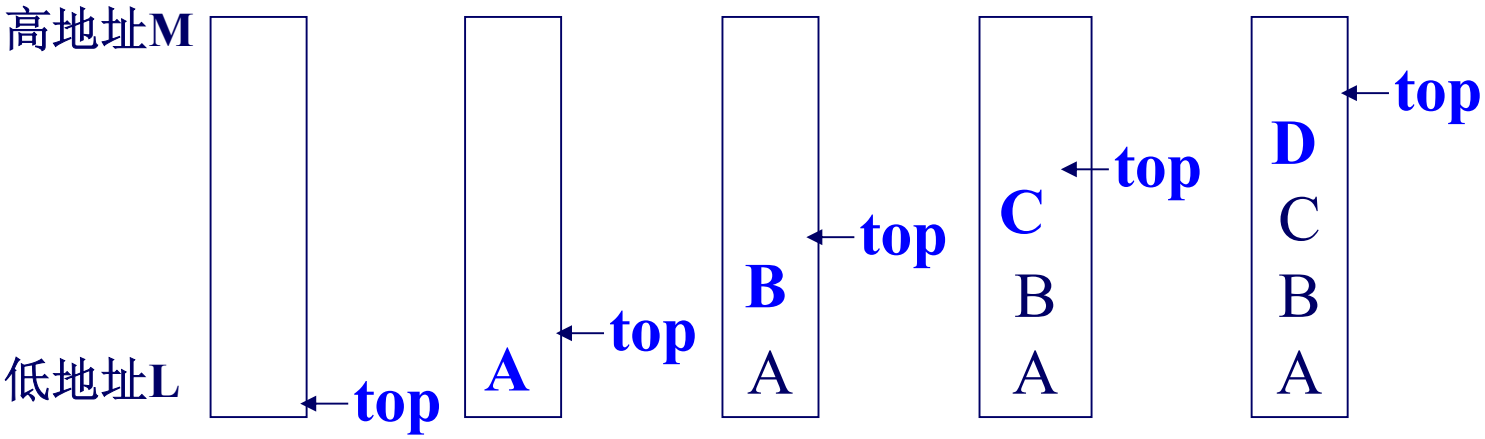
基本操作：

……

} ADT Stack

入栈、出栈、
建栈初始化、
判断栈满或栈空、
读栈顶元素值等。

顺序栈的入栈操作——例如用堆栈存放（A，B，C，D）



核心语句：

top=L;

Push (A);

Push (B);

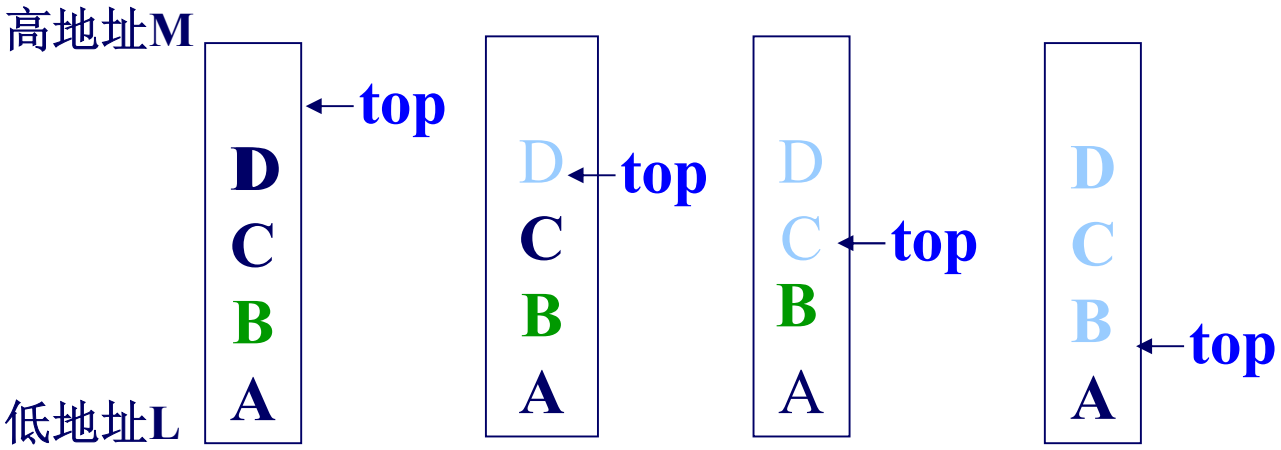
Push (C);

Push (D);

顺序栈入栈函数PUSH（）

```
status Push(ElemType e)
{ if(top>M) {上溢}
  else s[top++]=e;
}
```

顺序栈出栈操作——例如从栈中取出‘B’



核心语句：

Pop ( );

Pop ( );

Printf(Pop ());

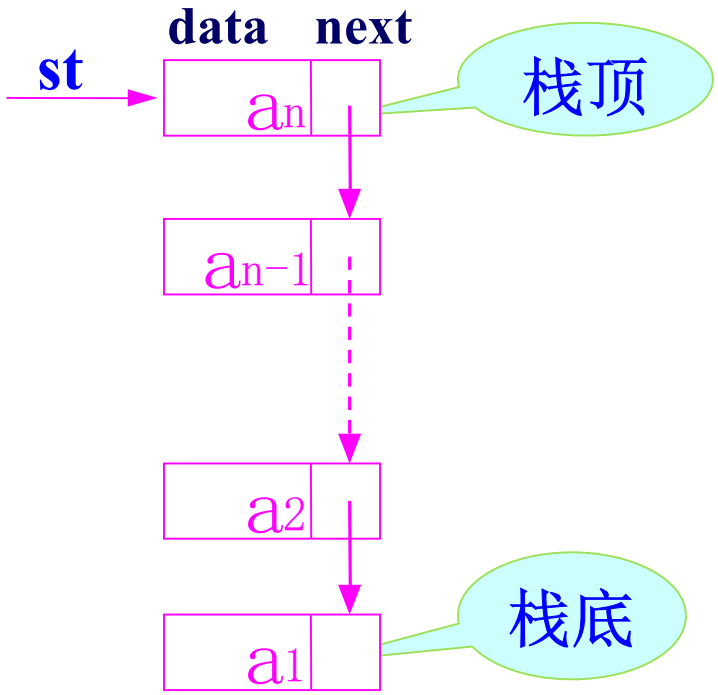
顺序栈出栈函数POP()

```
ElemType Pop( )
{ if(top=L) { 下溢 }
else { e=s[--top];
return(e);}
 }
```

链栈的入栈操作和出栈操作（教材省略）

栈也可以用链式结构来表示，用链式结构来表示的栈就是链栈

(1) 链栈的构造方式——以头指针为栈顶，在头指针处插入或删除.



链栈中每个结点由两个域构成：data域和next域，其定义为：

```
typedef Struct SNode{
        SElemType    data;
        Struct  SNode  * next;
    } Node;
```

```
Node  *st, *p;
int  m=sizeof(Node);
```

几点说明：

1）链栈不必设头结点，因为栈顶（表头）操作频繁；

2）链栈一般不会出现栈满情况，除非没有空间导致malloc( )分配失败。

3）链栈的入栈、出栈操作就是栈顶的插入与删除操作，修改指针即可完成。

4）采用链栈存储方式的优点是，可使多个栈共享空间；当栈中元素个数变化较大，且存在多个栈的情况下，链栈是栈的首选存储方式。

(2) 操作

链栈入栈函数

```
Push (SElemType e)
{ p=(Node*)malloc(m);
  if(!p){上溢}
else{ p->data=e; p->link=st; st=p;}
}
```

插入表头

链栈出栈函数

```
 ElemType Pop( )
{ if(st==NULL){下溢}
else{e=st->data;p=st;st=st->link;
        free(p);  return(e);}
 }
```

从表头删除

由此可以看出：一个链栈由其栈顶指针唯一指定。

设st指向栈顶元素，当st=NULL时表示栈空

栈的应用举例

例1：汉诺（Hanoi）塔————见教材P55

设计思路：用栈实现递归调用

例2：表达式求值 ————见教材P52

设计思路：用栈暂存运算符

例3： 数制转换（十转N）

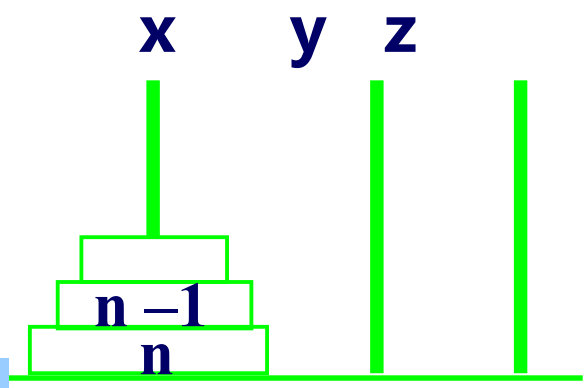
设计思路：用栈暂存低位值

例4：括号匹配的检验

设计思路：用栈暂存左括号

例1 汉诺（ Hanoi）塔

传说在创世纪时，在一个叫Brahma的寺庙里，有三个柱子，其中一柱上有64个盘子从小到大依次叠放，僧侣的工作是将这64个盘子从一根柱子移到另一个柱子上。



移动时的规则：

- **每次只能移动一个盘子；**
- **只能小盘子在大盘子上面；**
- **可以使用任一柱子。**

当工作做完之后，就标志着世界末日到来。

分析：

设三根柱子分别为 x, y, z , 盘子在 x 柱上，要移到 z 柱上。

1、当 n=1 时，盘子直接从 x 柱移到 z 柱上；

2、当 n>1 时，则：

① 设法将 前 n –1 个盘子 从 x 移到 y 柱上（**借助z**），则 盘子 n 就能从 x 移到 z 柱上；

② 再设法把n –1 个盘子 从 y 移到 z 柱上(**这又是一个与原来相同的问题，只是规模少１了**) 。

四川科技馆展项——梵天之塔

程序设计如下：

```
Void   Hanoi ( int n,  char x, char y, char z )
{ //将n个编号从上到下为1…n的盘子从x柱，借助y柱移到z柱
if ( n ==1 )
      move ( x , 1 , z ) ;  //将编号为1的盘子从x柱移到z柱
  else
  {
  Hanoi ( n-1 , x , z , y ) ;//先将n-1个盘从x柱借助z柱移到y柱
  move ( x ,  n,  z) ;          //将编号为n的盘子从x柱移到z柱
  Hanoi ( n-1 , y , x , z );   //再将n-1个编号从上到下为1…n-1的
          盘子从y柱借助x柱移到z柱
  }
} //Hanoi
```

递归运算！

例2 编写算法，用栈实现表达式3*（7－2）求值。

(表达式求值是栈应用的典型例子, 见P45)

注：本例采用 **“算符优先法”**。

一个算术表达式是由**操作数**（x，y，z...）和**算符**（*，/，+，-，（，），#）组成

表达式的起止符号

(1) 表达式求值必须满足算术四则运算规则：

a. 从左算到右

b. 先乘除，后加减

c. 先括号内，后括号外

教材P46中表3.1给出了算符之间的优先级

专为计算机处理而设计的表!

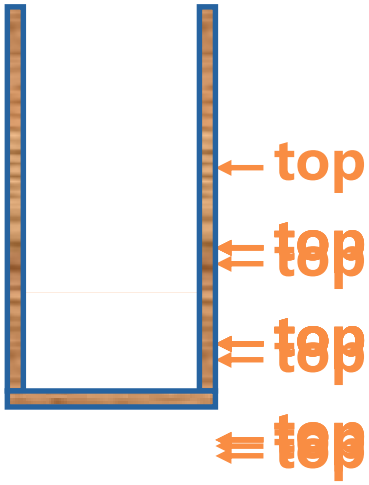
输入计算机的表达式为：# 3*（7－2）# 但是先看两个例子

✲ 前缀中缀和后缀表示方式

〖例子〗中缀表示: $a + b * c - d / e$

后缀表示: $a\ b\ c * + d\ e / -$

〖后缀表达式计算: 〗 $6\ 2 / 3 - 4\ 2 * + = 8$

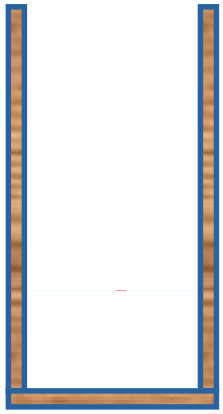


Get token: 6 (operand)	Get token: 2 (operand)
Get token: / (operator)	Get token: 3 (operand)
Get token: – (operator)	Get token: 4 (operand)
Get token: 2 (operand)	Get token: * (operator)
Get token: + (operator)	Pop: 8

〖中缀表达式转换为后缀表达式〗

$a * (b + c) / d = ?$

Output: $a\ b\ c\ +\ *\ d\ /$



← top

Get token: a (operand)	Get token: $*$ (times)
Get token: ((lparen)	Get token: b (operand)
Get token: + (plus)	Get token: c (operand)
Get token:) (rparen)	Get token: / (divide)
Get token: d (operand)	

$* \geq (?$

$* \geq / ?$

NO?!

为了实现算符优先算法，可以将以上两个步骤联合起来，方法就是设定两个工作栈：**OPTR**—存放运算符号，**OPND**—存放操作数或运算结果 。

operator & operand

(2) 算法思想：

1）首先让表达式的起始符#为运算符栈**OPTR**的栈底元素，并置操作数栈**OPND**为空栈；

示意图

2）依次读入表达式中的每个字符，

若运算符是‘#’且栈顶是‘#’，则结束计算，返回OPND栈顶值。

if（是操作数）→ 则PUSH（ OPND，操作数）；

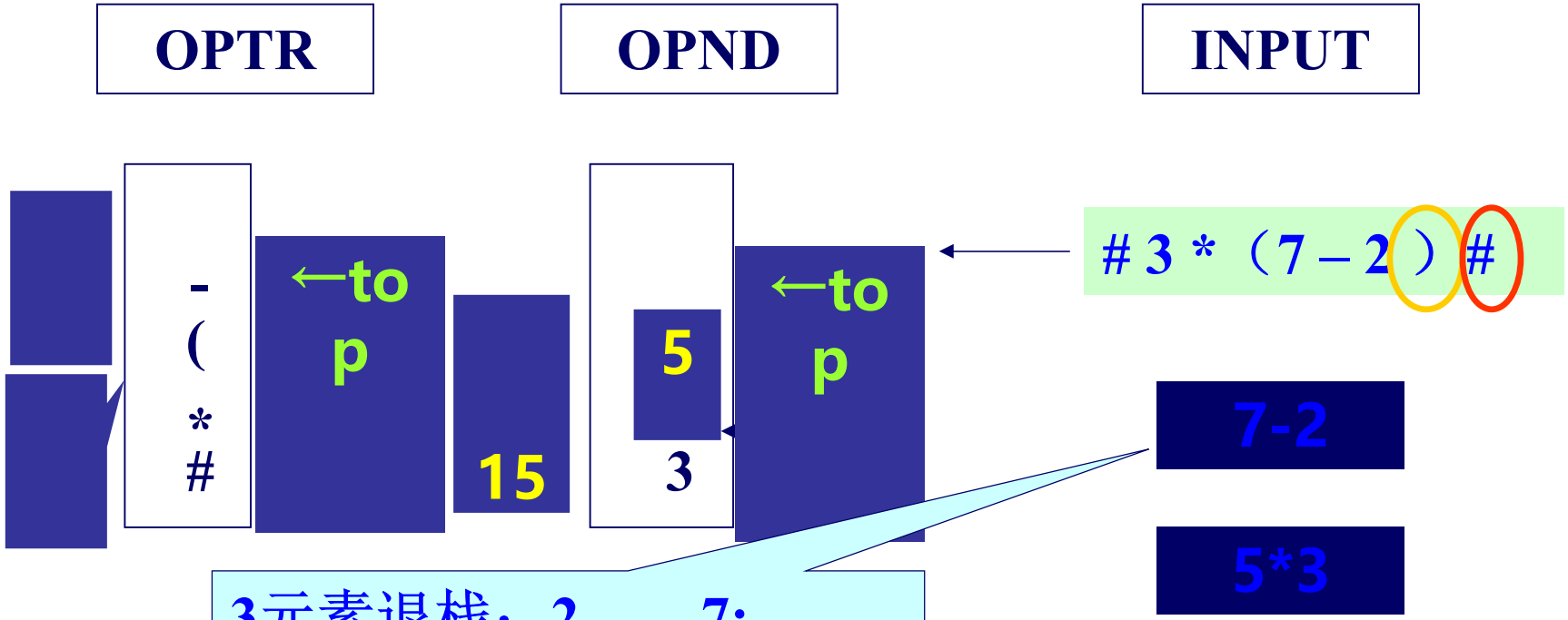
if（是运算符）→ 则与**OPTR**栈顶元素进行比较，按优先级**(详见P46表3.1的规定)** 进行操作；

if栈顶元素<输入算符，则算符压入OPTR栈，并接收下一字符

if栈顶元素=运算符但≠‘#’，则脱括号(弹出左括号)并收下一字；

if栈顶元素>运算符，则退栈、按栈顶计算，将结果压入OPND栈。且该未入栈的运算符要保留，继续与下一个栈顶元素比较！

表达式求值过程----机内运算示意图



3元素退栈：2， -，7;
计算：a, thata,b=7-2=5
将结果压入OPND栈
且运算符 ‘)’保留，继续与下一个栈顶元素比较！

P47 表达式求值过程的描述：　　# 3 *（7 – 2 ）#

OPTR	OPND	INPUT	OPERATE
#		3*(7-2)#	Push(opnd,’3’)
#	3	*(7-2)#	Push(optr,’*’)
#, *	3	(7-2)#	Push(optr,’(’)
#, *, (	3	7-2)#	Push(opnd,’7’)
#, *, (	3, 7	-2)#	Push(optr,’-’)
#, *, (, —	3, 7	2)#	Push(opnd,’2’)
#, *, (, —	3, 7, 2	)#	Operate(7-2)
#, *, (	3, 5	)#	Pop(optr)
#, *	3, 5	#	Operate(3*5)
#	15	#	GetTop(opnd)

程序见P46

```
Status   EvaluateExpression( OperandType  &result)  {
 InitStack(OPND); InitStack(OPTR);Push(OPTR ,'#');c=getchar();
  while((c!='#')||(GetTop(OPTR)!='#'))
  {  if (!In(c,OP)  { Push(OPND,c);   c=getchar();}
     else  switch(precede(GetTop(OPTR) , c))
          { case '<' :    Push(OPTR , c); c=getchar();  break;
            case '=':   Pop(OPTR);  c=getchar();  break;
           case '>' : Pop(OPTR ,theta); Pop(OPND,b);a=Pop();
                      Push(OPND, Operate(a, theta,b)) ;  break; ;
          } //switch
  } //while
  result=GetTop(OPND);
}//EvaluateExpression
```

栈顶与运算符比较并查表3.1

判C是否操作符

Operate=a θ b

问：教材P46表3.1中，θ1和θ2哪个对应栈顶元素，哪个对应键盘输入值?

答：根据P46 Precede()函数可知，θ1对应栈顶元素

附：

- 由表3.1可看出，右括号) 和井号 # 作为θ2时级别最低；
- 由c 规则得出：<u>*，/，+，-</u>为θ1时的优先权低于‘（’，高于‘）’
- 由a规则得出：‘（’=‘）’ 表明括号内的运算已完成；‘#’=‘#’ 表明表达式求值完毕。